附录 F｜完整命令参考（无省略号）

NM6008 Module 6 — Linux/CentOS 7 完整分阶段命令手册

```
Cadence Innovus 20.12 · GF22 FDSOI · 基于课程 2026 Manual + 教师更新 + 本次实验 CHIP.tcl

说明:
1. 【Linux Shell】和【Innovus Tcl】两个提示符不可混用。
2. 这是按阶段执行的模板，不是建议一次 source 全文的 Tcl 脚本；每个阶段必须确认运行结果。
3. 你本机老师给的 CHIP.tcl、Route.tcl 是具体版本的最终准绳；不匹配时先检查，不能盲目运行。
4. 老师最新通知: Special Route 使用 M1(1) / C2(4)；第 13 步 Parasitic Extraction 免做。
5. 不重复运行 addIoFiller、addEndCap、place_design、Route.tcl、addFiller，也不要覆盖唯一可恢复 checkpoint。
```

A. 工具启动与输入检查 【Linux Shell: ...$】

```
cd ~/NM6008/Module6
pwd
ls -lh cshrc_inno Syn_MatrixMultiplier_IO.globals Syn_MatrixMultiplier_IO.view
ls -lh scripts/CHIP.tcl scripts/Route.tcl
ls -ld IO LEF LIB GDS RC Netlist timingReports save_design
csh
source cshrc_inno
which innovus
innovus

若新会话要恢复已完成的设计，不要 source CHIP.tcl；先查看:
【Linux Shell】ls -lh save_design/NM6008_DRC_CLEAN save_design/NM6008_DRC_CLEAN.dat
【Innovus Tcl】source save_design/NM6008_DRC_CLEAN
【Innovus Tcl】dbGet top.name
预期顶层: Syn_MatrixMultiplier_IO
```

B. 原 Manual 1-15 步 【Innovus Tcl: innovus N>】

01 | Import Design / 设计导入（Manual P4-5）

```
source Syn_MatrixMultiplier_IO.globals
init_design
# 成功标准: IO Pads/Core 出现，顶层和库正常加载。不要重复运行 init_design。
```

02 | Global Net Connect / 全局电源网连接（P5）

```
clearGlobalNets
globalNetConnect vdd -type pgpin -pin VDD -inst *
globalNetConnect gnd -type pgpin -pin VSS -inst *
# GUI: Power → Connect Global Nets; 检查 vdd:VDD 和 gnd:VSS。
```

03 | Floorplanning（P6-7）

```
getIoFlowFlag
setIoFlowFlag 0
floorPlan -site GF22_SST -d 1400 1400 150 150 150 150 -noSnapToGrid
# 确认 Die/Core/Pad；仅适用于本次指定尺寸和 GF22_SST Site。
```

04 | Power Ring（P7-9）

```
setAddRingMode -ring_target default -extend_over_row 0 \
  -ignore_rows 0 -avoid_short 0 -skip_crossing_trunks none \
  -stacked_via_top_layer LB -stacked_via_bottom_layer M1 \
  -via_using_exact_crossover_size 1 -orthogonal_only true \
  -skip_via_on_pin {standardcell} -skip_via_on_wire_shape {noshape}
addRing -nets {gnd vdd} -type core_rings -follow core \
  -layer {top C2 bottom C2 left C1 right C1} \
  -width {top 1.8 bottom 1.8 left 1.8 right 1.8} \
  -spacing {top 1.8 bottom 1.8 left 1.8 right 1.8} \
  -offset {top 1.8 bottom 1.8 left 1.8 right 1.8} \
  -center 0 -threshold 0 -jog_distance 0 \
  -snap_wire_center_to_grid None
# GUI: Power → Power Planning → Add Ring。检查 gnd/vdd Ring 及层名。
```

05 | Special Net Routing / SRoute（P9-10）

```
# 老师 2026 更新：原 Manual 的 MET1(1)，MET4(4) 改为 M1(1)，C2(4)，包括下面三处。
setSrouteMode -viaConnectToShape {noshape}
sroute -connect {blockPin padPin padRing corePin floatingStripe} \
  -layerChangeRange {M1(1) C2(4)} \
  -blockPinTarget {nearestTarget} \
  -padPinPortConnect {allPort oneGeom} \
  -padPinTarget {nearestTarget} \
  -corePinTarget {firstAfterRowEnd} \
  -floatingStripeTarget {blockring padring ring stripe ringpin blockpin followpin} \
  -allowJogging 1 \
  -crossoverViaLayerRange {M1(1) C2(4)} \
  -nets {gnd vdd} -allowLayerChange 1 \
  -blockPin useLef -targetViaLayerRange {M1(1) C2(4)}
# GUI: Route → Special Route; Nets=gnd vdd; BlockPins/PadPins/PadRing/CorePins/FloatingStripe。
# 以下拉伸命令仅供当前固定 Floorplan/坐标下逐条复核，不能原样用于新尺寸：
# selectWire 628.9500 1354.0000 633.0500 1354.1180 3 gnd
# uiSetTool stretchWire
# editResize -direction y -offset -148.259 -side low -keep_center_line auto
# deselectAll
# selectWire 742.9500 1354.0000 747.0500 1354.1180 3 vdd
# editResize -direction y -offset -148.384 -side low -keep_center_line auto
# editTrim
# deselectAll
# selectWire 748.9500 45.8820 753.0500 46.0000 3 gnd
# editResize -direction y -offset 148.524 -side high -keep_center_line auto
# editTrim
# deselectAll
```

06 | IO Pad Filler Placement（P11）

```
# 从大到小填充；N/S = _V，W/E = _H。
addIoFiller -cell "IN22FDX_GPIO18_10M19S40PI_FILL20_V IN22FDX_GPIO18_10M19S40PI_FILL10_V IN22FDX_GPIO18_10M19S40PI_FILL5_V
IN22FDX_GPIO18_10M19S40PI_FILL1_V" -prefix IOFILLER_N -side n
addIoFiller -cell "IN22FDX_GPIO18_10M19S40PI_FILL20_V IN22FDX_GPIO18_10M19S40PI_FILL10_V IN22FDX_GPIO18_10M19S40PI_FILL5_V
IN22FDX_GPIO18_10M19S40PI_FILL1_V" -prefix IOFILLER_S -side s
addIoFiller -cell "IN22FDX_GPIO18_10M19S40PI_FILL20_H IN22FDX_GPIO18_10M19S40PI_FILL10_H IN22FDX_GPIO18_10M19S40PI_FILL5_H
IN22FDX_GPIO18_10M19S40PI_FILL1_H" -prefix IOFILLER_W -side w
addIoFiller -cell "IN22FDX_GPIO18_10M19S40PI_FILL20_H IN22FDX_GPIO18_10M19S40PI_FILL10_H IN22FDX_GPIO18_10M19S40PI_FILL5_H
IN22FDX_GPIO18_10M19S40PI_FILL1_H" -prefix IOFILLER_E -side e
# 观察每个侧边 IO filler 是否填满间隙。
```

07 | End Cap Placement（P11-12）

```
snapFPlan -all
set CELL_PREFIX "HDBSLT20"
setEndCapMode -reset
setEndCapMode \
  -topEdge "${CELL_PREFIX}_CAPB1" \
  -bottomEdge "${CELL_PREFIX}_CAPT1" \
  -leftEdge "${CELL_PREFIX}_CAPR9_1" \
  -rightEdge "${CELL_PREFIX}_CAPL9_1" \
  -rightTopCorner "${CELL_PREFIX}_CAPBOUCL9_1" \
  -rightBottomCorner "${CELL_PREFIX}_CAPTOUCL9_1" \
  -leftTopCorner "${CELL_PREFIX}_CAPBOUCR9_1" \
  -leftBottomCorner "${CELL_PREFIX}_CAPTOUCR9_1" \
  -leftBottomEdge "${CELL_PREFIX}_CAPTLINC9_1" \
  -lefttopEdge "${CELL_PREFIX}_CAPBLINC9_1" \
  -rightBottomEdge "${CELL_PREFIX}_CAPTRINC9_1" \
  -righttopEdge "${CELL_PREFIX}_CAPBRINC9_1" \
  -fitGap true
addEndCap -prefix ENDCAP
# 检查 GF22 库中确实存在这些 cell；不要复制 Manual 排版中的尾随空格。
```

08 | Standard Cell Placement（P13）

```
setDesignMode -process 22
setPlaceMode -reset
setPlaceMode -congEffort auto -timingDriven 1 -modulePlan 1 -clkGateAware 1 -powerDriven 0 -ignoreScan 1 -reorderScan 1
-ignoreSpare 0 -placeIOPins 0 -moduleAwareSpare 0 -maxDensity 0.8 -preserveRouting 1 -rmAffectedRouting 0 -checkRoute 0
-swapEEQ 0
setPlaceMode -place_detail_color_aware_legal true
setPlaceMode -place_detail_check_cut_spacing true
setPlaceMode -place_detail_use_check_drc true
setPlaceMode -fp false
place_design
checkPlace CHIP_TOP.checkPlace
saveDesign save_design/MatrixMultiplier_post_place
# GUI: Place → Place Standard Cell；确认 Placement 已成功，不能只看密度。
```

09 | Pre-CTS Timing + Optimization + CTS（P14-17）

```
timeDesign -preCTS -idealClock -pathReports -drvReports -slackReports -numPaths 50 -prefix Syn_MatrixMultiplier_IO_preCTS
-outDir timingReports
setOptMode -fixCap true -fixTran true -fixFanoutLoad false
```

```
optDesign -preCTS
# 下面两步来自本次 CHIP.tcl 的额外 optimization（仅按老师提供脚本使用）：
setOptMode -fixCap false -fixTran false -fixFanoutLoad false
optDesign -preCTS -incr
setOptMode -fixCap false -fixTran false -fixFanoutLoad true
optDesign -preCTS -drv
# 构建真实时钟树：
create_ccopt_clock_tree_spec
ccopt_design -prefix cts
ctd_win -id ctd_window
# Post-CTS Setup/Hold 分阶段优化与复测：
timeDesign -postCTS -pathReports -drvReports -slackReports -numPaths 50 -prefix CHIP_postCTS -outDir timingReports
setOptMode -fixCap true -fixTran true -fixFanoutLoad true
optDesign -postCTS
timeDesign -postCTS -hold -pathReports -slackReports -numPaths 50 -prefix CHIP_postCTS -outDir timingReports
setOptMode -fixCap true -fixTran true -fixFanoutLoad true
optDesign -postCTS -hold
optDesign -postCTS -drv
# 必须复测，禁止仅凭中间 WNS 判断修复成功：
timeDesign -postCTS -pathReports -drvReports -slackReports -numPaths 50 -prefix final_postCTS -outDir timingReports
timeDesign -postCTS -hold -pathReports -slackReports -numPaths 50 -prefix final_postCTS -outDir timingReports
report_ccopt_clock_trees -file clock_trees_Post_CTS.rpt
saveDesign save_design/NM6008_postCTS_final
# Hold 从 -2.492 ns / 53 paths 逐步收敛；本次最终 Post-Route Hold WNS +0.419 ns。
```

10 | NanoRoute / Detail Routing（P17-19）

```
# 教师提供的 Route.tcl 内部含有真正的路由命令；先检查其内容，确保未重复运行。
# 【Linux Shell，在另一个普通终端】sed -n '1,220p' scripts/Route.tcl
source ./scripts/Route.tcl
# Routing 后的 Post-Route Setup/Hold，输出正负 slack 与 DRV：
timeDesign -signoff -pathReports -drvReports -slackReports -numPaths 50 -prefix final_route -outDir timingReports
timeDesign -signoff -hold -pathReports -slackReports -numPaths 50 -prefix final_route -outDir timingReports
# 注意：命令 -signoff 本身不能证明外部 Quantus 已完成寄生提取。
```

11 | Core Filler + DRC（P19-20）

```
getFillerMode -quiet
addFiller -cell HDBSLT20_FILL64 HDBSLT20_FILL32 HDBSLT20_FILL16 HDBSLT20_FILL8 HDBSLT20_FILL4 HDBSLT20_FILL3 HDBSLT20_FILL2
HDBSLT20_FILL1 -prefix FILLER -doDRC
# 真实实验曾因 filler 后 DRC 19→1→0 进行针对性 ECO；建议先保存：
saveDesign save_design/NM6008_before_DRC_repair
verify_drc -limit 0 -report DRC_before_repair.rpt
# 只在实际存在 DRC 违例，且确定当前设计允许 ECO 时执行以下命令：
# ecoRoute -fix_drc
# verify_drc -limit 0 -report DRC_after_ECO.rpt
# 最终确认 no DRC violations 之后才保存：
# saveDesign save_design/NM6008_DRC_CLEAN
# 不要对已修复且已输出最终文件的设计无条件重复修复。
```

12 | GDSII Export（P20-21）

```
streamOut Syn_MatrixMultiplier_IO.gds2 \
  -mapFile GDS/22FDSOI_10M_2Mx_5Cx_1Jx_2Qx_LB_edi2gds_colored.layermap \
  -libName DesignLib -structureName Syn_MatrixMultiplier_IO \
  -units 1000 -mode ALL
```

13 | Parasitic Extraction（P21）— 2026 教师免做

```
# 老师通知「Parasitic Extraction no need to do」；此环节跳过。
# 仅用于辨识旧 Manual，切勿报告 SPEF 成功：
# rcOut -spef Syn_MatrixMultiplier_IO.spef -rc_corner typical
# 若看到 GLIBC_2.25/2.27/2.28 not found: Quantus 24.10 与 CentOS 7 环境不匹配，不要手改 PDK 二进制文件。
```

14 | SDF Output（P21）

```
all_hold_analysis_views
all_setup_analysis_views
write_sdf Syn_MatrixMultiplier_IO.sdf
```

15 | Post-Layout Netlist Output（P22）

```
saveNetlist output_netlist/Syn_MatrixMultiplier_IO.v
# 输出完成建议再保存 checkpoint:
saveDesign save_design/NM6008_final_exported
```

C. 最终验收与快速诊断 【Linux Shell: ...$】

```
cd ~/NM6008/Module6
ls -lh Syn_MatrixMultiplier_IO.gds2 Syn_MatrixMultiplier_IO.sdf output_netlist/Syn_MatrixMultiplier_IO.v
cat DRC_final_check.rpt
zcat timingReports/NM6008_final.summary.gz
zcat timingReports/NM6008_final_hold.summary.gz
grep -nE '^module[[:space:]]+Syn_MatrixMultiplier_IO([[:space:]]|\(|#)' output_netlist/Syn_MatrixMultiplier_IO.v
# 如果报告文件名不一致，请先查看现有文件，不要把示例路径当成必然存在:
ls -lh timingReports/*summary.gz
# 常见报错关键词搜索:
grep -Ein 'ERROR|failed|GLIBC|NREX-94|LS_PLACEOPT|IMPEXT|IMPTS-16|Killed' innovus.log*

正确的本次最终记录: Setup WNS +87.329 ns; Hold WNS +0.419 ns; Timing Violations 0;
DRC report: No DRC violations were found; GDSII / SDF / Gate Netlist 已输出。
这属于课程实验结果，不应写成外部完整制造签核。
```

D. 报错速查

```
innovus: Command not found         → 切 csh / source cshrc_inno / which innovus
setExtractRCMode: Command not found→ 检查是否在 Linux shell，应该进入 innovus N>
report_saif: Command not found     → 属于 Synopsys 工具域，不是普通 shell/Innovus 指令
restoreDesign: Command not found   → 对本机的保存脚本用 source save_design/CHECKPOINT
GLIBC_2.25/2.27/2.28 not found  → Quantus 新版与 CentOS 7 旧 libc 不兼容，按更新免做 Section 13
LS_PLACEOPT::runCurrentState       → 联动 Quantus/RC 失败、路径/环境/数据库状态，先检查日志与依赖
NREX-94 11 routing layers / 1 mapped → 检查 QRC 物理层映射与对应 tech，不要把二进制 qrcTechFile 当文本改
Hold WNS 为负                   → Post-CTS check → optDesign -postCTS -hold → Post-Route 重新检查
19 / 1 DRC Violation           → verify_drc 定位 Short/Spacing → 保存检查点 → 针对性 ECO → 再检查
verifyConnectivity 输出 Killed  → 可能遭 OS OOM killer，检查内存/日志、避免无约束重复运行
GDS 为空或尺寸不对             → 检查 streamOut mapFile、libName、structureName、错误摘要
SDF 不存在                     → 在 Innovus 检查 write_sdf 的输出与时序视图

请配合完整中文图文 PDF 使用: PDF 包含 15 个步骤的原 Manual GUI 图、真实机房截图和 18 个 Debug 案例。
```